Universidad Peruana de Ciencias Aplicadas

**Arquitectura de Aplicaciones Web**

---

## Taller S2 - API REST: Catálogo de Videojuegos por Capas

**Objetivo:**

Construir una API REST en Spring Boot para gestionar un catálogo de videojuegos, aplicando arquitectura en capas: Entidad, Repositorio, Servicio y Controlador.

**Duración:** 2 horas

**Escenario:**

Eres desarrollador backend de una tienda de videojuegos en línea. El equipo de frontend ya está listo para consumir tu API. Debes implementar el backend con Spring Boot que permita listar, agregar, actualizar y eliminar videojuegos del catálogo.

**Requisitos Previos:**

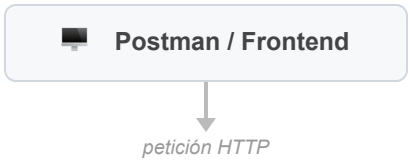
- JDK 17 o superior instalado
- IntelliJ IDEA con soporte para proyectos Maven
- Postman instalado
- Anotaciones repasadas en clase: **@RestController, @Service, @Repository, @PathVariable, @RequestBody**

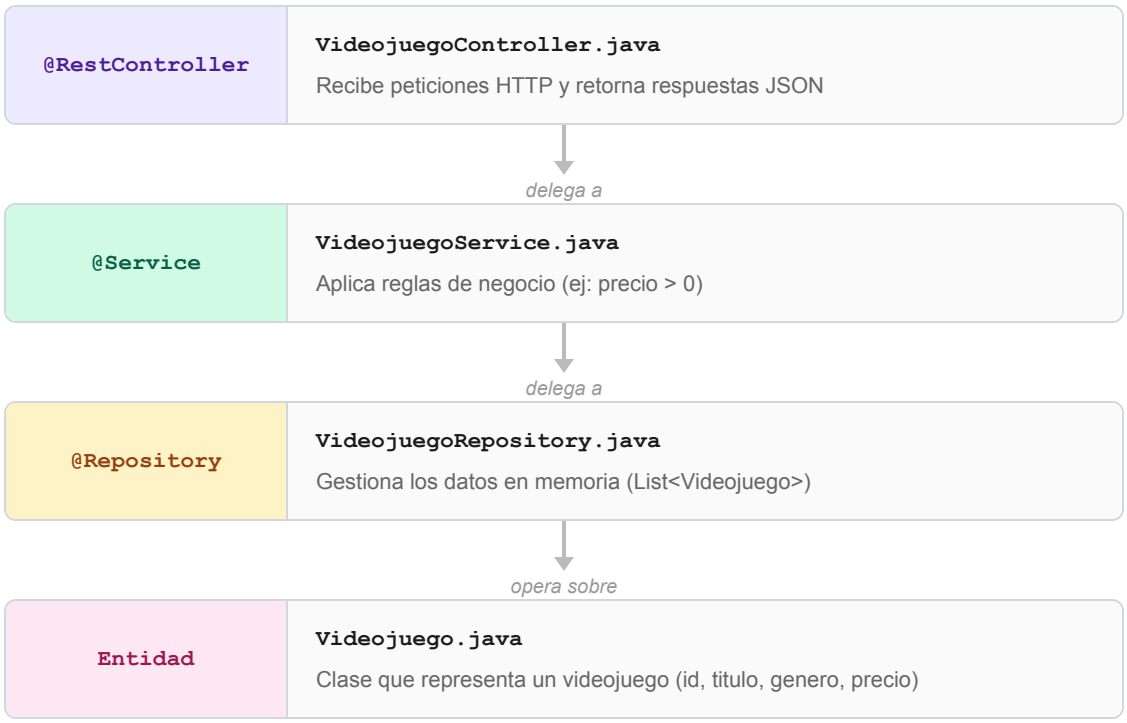
**Paso previo - Crear el proyecto base:**

1. Ve a **https://start.spring.io** y genera un proyecto con: Project: **Maven** · Language: **Java** · Spring Boot: **3.x.x** · Group: **com.miempresa** · Artifact: **videojuegos-api**.
2. Agrega las dependencias: **Spring Web** y **Spring Boot DevTools**.
3. Descarga, descomprime y abre el proyecto en **IntelliJ IDEA**.
4. Ejecuta la aplicación y verifica que corre en **http://localhost:8080**.

### Arquitectura del Sistema

La API sigue el flujo de capas visto en clase. Cada capa tiene una única responsabilidad:

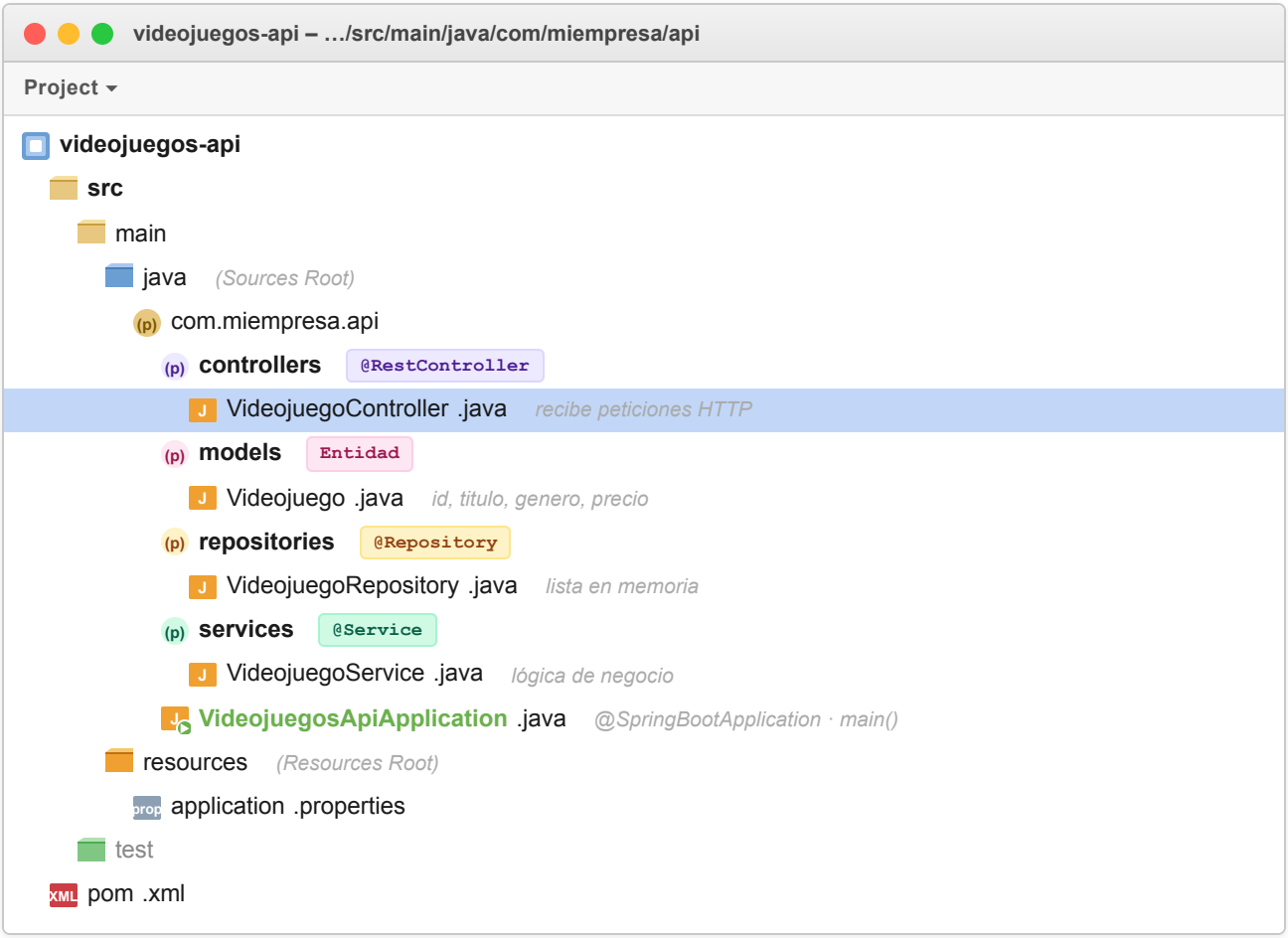





> **Regla fundamental:** El Controlador no conoce el Repositorio. El Repositorio no conoce el Controlador. La comunicación siempre pasa por el Servicio.

### Estructura del proyecto en IntelliJ IDEA

Al finalizar todos los ejercicios, la estructura de paquetes debe verse así:



## Ejercicio 1 - Entidad `models/Videojuego.java`

**Objetivo:** Crear la clase **Videojuego.java** que represente un videojuego del catálogo con sus atributos.

**Instrucciones:**

1. Dentro de **src/main/java/com/miempresa/api/**, crea el paquete **models**.
2. Crea la clase **Videojuego.java** con los siguientes atributos:
   - **id** (Long)
   - **titulo** (String) - nombre del videojuego
   - **genero** (String) - Acción, RPG, Deportes, etc.
   - **precio** (double) - precio en soles
3. Genera un constructor con los cuatro parámetros y un constructor vacío.
4. Genera los métodos **getter** y **setter** para cada atributo (en IntelliJ: clic derecho -> Generate).

**`// Videojuego.java - completa los espacios en blanco`**

```
// El paquete indica dónde vive esta clase dentro del proyecto
package com.miempresa.api.models;

// La entidad NO necesita anotaciones de Spring - es solo un modelo de datos
public class Videojuego {

    // Atributos privados: solo accesibles mediante getters y setters
    private ________ id;       // identificador único (tipo numérico entero grande)
    private ________ titulo;   // nombre del videojuego (tipo texto)
    private ________ genero;   // categoría: Acción, RPG, Deportes... (tipo texto)
    private ________ precio;   // precio en soles (tipo decimal)

    // Constructor vacío: requerido por Spring para deserializar el JSON del body
    public Videojuego() { }

    // Constructor completo: se usa en el repositorio para crear datos iniciales
    public Videojuego(________ id, ________ titulo,
                      ________ genero, ________ precio) {
        this.id     = id;      // "this.id" = atributo de la clase, "id" = parámetro
        this.titulo = titulo;
        this.genero = genero;
        this.precio = precio;
    }

    // Getters: permiten leer los atributos desde fuera de la clase
    // Setters: permiten modificar los atributos desde fuera de la clase
```

```
    // En IntelliJ: clic derecho -> Generate -> Getter and Setter
}
```

**Resultado esperado: Clase Videojuego con 4 atributos, constructor completo y métodos getter/setter.**

---

## Ejercicio 2 - Repositorio `repositories/VideojuegoRepository.java`

**Objetivo:** Crear **VideojuegoRepository.java** que almacene los videojuegos en una lista en memoria y exponga métodos para leer, guardar, actualizar y eliminar.

**Instrucciones:**

1. Crea el paquete **repositories** dentro de **com.miempresa.api**.
2. Crea la clase **VideojuegoRepository.java** anotada con **@Repository**.
3. Declara una **List<Videojuego>** privada e inicialízala en el constructor con al menos 3 videojuegos de ejemplo.
4. Implementa los métodos indicados.

**// VideojuegoRepository.java - completa los espacios en blanco**

```
package com.miempresa.api.repositories;

import com.miempresa.api.models.Videojuego;
import org.springframework.stereotype.Repository;
import java.util.*;

// @Repository: le dice a Spring que esta clase gestiona el acceso a datos
// Spring la registra como un bean y permite inyectarla en otras clases
@________
public class VideojuegoRepository {

    // La lista simula una base de datos en memoria
    // Es privada: solo este repositorio puede acceder a ella directamente
    private List<Videojuego> catalogo = new ArrayList<>();
    private Long nextId = 1L; // contador para asignar IDs únicos y crecientes

    // El constructor carga datos de ejemplo al iniciar la aplicación
    public VideojuegoRepository() {
        catalogo.add(new Videojuego(nextId++, "The Legend of Zelda", "Aventura",  199.9
        catalogo.add(new Videojuego(nextId++, "FIFA 25",             "Deportes",  249.9
        catalogo.add(new Videojuego(nextId++, "Elden Ring",          "RPG",       179.9
    }

    // Retorna todos los videojuegos de la lista
```

```
    public List<Videojuego> findAll() {
        return ________; // ¿qué variable contiene todos los videojuegos?
    }

    // Busca un videojuego por su id usando Stream
    // Optional evita retornar null: si no existe, retorna Optional.empty()
    public Optional<Videojuego> findById(Long id) {
        return catalogo.stream()
                .filter(v -> v.getId().________(id)) // ¿qué método compara dos Long?
                .findFirst();
    }

    // Guarda un nuevo videojuego asignándole el próximo id disponible
    public Videojuego save(Videojuego v) {
        v.setId(________); // asigna nextId y luego incrementa en 1
        catalogo.add(v);
        return v; // retorna el videojuego ya con su id asignado
    }

    // Actualiza los datos si el id existe; si no, retorna Optional.empty()
    public Optional<Videojuego> update(Long id, Videojuego datos) {
        return findById(id).map(v -> { // map solo se ejecuta si el Optional tiene valo
            v.setTitulo(datos.________()); // obtiene el título del objeto recibido
            v.setGenero(datos.________());
            v.setPrecio(datos.________());
            return v;
        });
    }

    // Elimina el videojuego con ese id; retorna true si lo encontró y eliminó
    public boolean deleteById(Long id) {
        return catalogo.removeIf(v -> v.getId().________(id));
    }
}
```

**Resultado esperado: Repositorio con lista en memoria, datos iniciales y métodos CRUD usando Optional para evitar valores nulos.**

---

## Ejercicio 3 - Servicio `services/VideojuegoService.java`

**Objetivo:** Crear **VideojuegoService.java** que reciba el repositorio por inyección de dependencias y aplique la siguiente regla de negocio: *no se puede registrar un videojuego con precio menor a 1.0*.

**Instrucciones:**

1. Crea el paquete **services** y dentro la clase **VideojuegoService.java**.

2. Anota la clase con **@Service**.
3. Inyecta **VideojuegoRepository** a través del constructor (no usar @Autowired en el campo).
4. Implementa los métodos delegando al repositorio.
5. En **agregar()**, valida que el precio sea mayor o igual a 1.0 antes de guardar.

**// VideojuegoService.java - completa los espacios en blanco**

```
package com.miempresa.api.services;

import com.miempresa.api.models.Videojuego;
import com.miempresa.api.repositories.VideojuegoRepository;
import org.springframework.stereotype.Service;
import java.util.*;

// @Service: marca esta clase como capa de lógica de negocio
// Spring la registra y permite inyectarla en el controlador
@________
public class VideojuegoService {

    // "final" garantiza que la referencia al repositorio no cambia después de asignarse
    private final VideojuegoRepository repository;

    // Inyección por constructor: Spring detecta el parámetro y pasa el bean automáticam
    // Es la forma recomendada - más clara y fácil de testear que @Autowired en el campo
    public VideojuegoService(________ repository) {
        this.repository = repository;
    }

    // Delega directamente al repositorio - no hay lógica adicional aquí
    public List<Videojuego> listar() {
        return repository.________(); // ¿qué método del repositorio lista todo?
    }

    // Retorna Optional: el controlador decidirá si responde 200 o 404
    public Optional<Videojuego> buscarPorId(Long id) {
        return repository.________(id);
    }

    // Aquí vive la regla de negocio: el servicio valida ANTES de guardar
    public Videojuego agregar(Videojuego v) {
        if (v.getPrecio() < ________) { // ¿cuál es el precio mínimo permitido?
            // IllegalArgumentException genera una respuesta de error que el
            // controlador puede capturar con @ExceptionHandler
            throw new IllegalArgumentException(
                "El precio del videojuego debe ser mayor a 0");
        }
```

```
        return repository._________(v); // ¿qué método del repositorio guarda un nuevo ob
    }

    // Delega la actualización; si el id no existe, el repositorio retorna Optional.emp
    public Optional<Videojuego> actualizar(Long id, Videojuego v) {
        return repository._________(id, v);
    }

    // Retorna boolean para que el controlador sepa si encontró el videojuego o no
    public boolean eliminar(Long id) {
        return repository._________(id);
    }
}
```

**Resultado esperado: Servicio que delega al repositorio y rechaza videojuegos con precio inválido antes de guardarlos.**

---

## Ejercicio 4 - Controlador `controllers/VideojuegoController.java`

**Objetivo:** Crear **VideojuegoController.java** que exponga los endpoints REST y delegue toda la lógica al Servicio. Debe retornar los códigos HTTP correctos usando **ResponseEntity**.

**Instrucciones:**

1. Crea el paquete **controllers** y la clase **VideojuegoController.java**.
2. Anota la clase con **@RestController** y **@RequestMapping("/api/videojuegos")**.
3. Inyecta **VideojuegoService** por constructor.
4. Implementa los 5 endpoints. Usa **ResponseEntity** en los métodos que pueden retornar 404.
5. Agrega un **@ExceptionHandler** para capturar **IllegalArgumentException** y retornar 400.

**// VideojuegoController.java - completa los espacios en blanco**

```
package com.miempresa.api.controllers;

import com.miempresa.api.models.Videojuego;
import com.miempresa.api.services.VideojuegoService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

// @RestController = @Controller + @ResponseBody
// Indica que esta clase maneja peticiones HTTP y retorna JSON automáticamente
@________
// Define la URL base para todos los endpoints de esta clase
@RequestMapping("/api/videojuegos")
```

```
public class VideojuegoController {

    private final VideojuegoService service;

    // Spring inyecta el ServicioService automáticamente por constructor
    public VideojuegoController(________ service) {
        this.service = service;
    }

    // GET /api/videojuegos -> retorna la lista completa (siempre 200 OK)
    @GetMapping
    public List<Videojuego> listar() {
        return service.________(); // ¿qué método del servicio lista todo?
    }

    // GET /api/videojuegos/{id} -> retorna 200 con el videojuego o 404 si no existe
    // @PathVariable extrae el {id} de la URL y lo convierte a Long
    @________ ("/{id}")
    public ResponseEntity<Videojuego> obtener(@________ Long id) {
        return service.buscarPorId(id)
                .map(ResponseEntity::ok)          // si existe -> 200 OK con el objeto
                .orElse(ResponseEntity.________.build()); // si no existe -> 404 Not Fo
    }

    // POST /api/videojuegos -> crea un nuevo videojuego con los datos del body JSON
    // @RequestBody convierte el JSON recibido en un objeto Videojuego
    @PostMapping
    public ResponseEntity<?> agregar(@________ Videojuego v) {
        return ResponseEntity.ok(service.________(v)); // delega al servicio y retorna
    }

    // PUT /api/videojuegos/{id} -> actualiza los datos del videojuego con ese id
    @________ ("/{id}")
    public ResponseEntity<Videojuego> actualizar(
            @PathVariable Long id,      // id de la URL
            @RequestBody Videojuego v) { // datos nuevos del body
        return service.actualizar(id, v)
                .map(ResponseEntity::ok)              // encontró y actualizó -> 200
                .orElse(ResponseEntity.notFound().build()); // no existe -> 404
    }

    // DELETE /api/videojuegos/{id} -> elimina el videojuego y confirma con mensaje
    @DeleteMapping("/{id}")
    public ResponseEntity<String> eliminar(@PathVariable Long id) {
        if (service.________(id)) { // retorna true si el videojuego existía
            return ResponseEntity.ok("Videojuego eliminado del catálogo");
```

```
        }
        // notFound() no permite body, por eso usamos status(404)
        return ResponseEntity._________.body("Videojuego no encontrado");
    }

    // @ExceptionHandler captura automáticamente la excepción lanzada en el servicio
    // Evita que Spring devuelva un error 500 genérico; retorna 400 con mensaje claro
    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<String> manejarError(IllegalArgumentException ex) {
        return ResponseEntity._________(ex.getMessage()); // ¿qué status HTTP indica dat
    }
}
```

**Resultado esperado: Controlador con 5 endpoints, manejo de 404 con ResponseEntity y captura de errores de validación con 400.**

---

## Ejercicio 5 - Pruebas con Postman

**Objetivo:** Verificar que todos los endpoints funcionan correctamente. Base URL: **http://localhost:8080**

| Método | URL | Body (JSON) | Respuesta esperada |
|---|---|---|---|
| **GET** | `/api/videojuegos` | - | 200 · Lista de 3 videojuegos |
| **GET** | `/api/videojuegos/1` | - | 200 · The Legend of Zelda |
| **GET** | `/api/videojuegos/99` | - | 404 Not Found |
| **POST** | `/api/videojuegos` | {"titulo":"Minecraft","genero":"Sandbox","precio":89.90} | 200 · Videojuego con id=4 |
| **POST** | `/api/videojuegos` | {"titulo":"Gratis","genero":"Acción","precio":0} | 400 · "El precio debe ser mayor a 0" |
| **PUT** | `/api/videojuegos/2` | {"titulo":"FIFA 26","genero":"Deportes","precio":279.90} | 200 · FIFA actualizado |
| **DELETE** | `/api/videojuegos/3` | - | 200 · "Videojuego |

| Método | URL | Body (JSON) | Respuesta esperada |
|---|---|---|---|
| | | | eliminado del catálogo" |

**Resultado esperado: La API responde correctamente a todos los casos, incluyendo 404 para IDs inexistentes y 400 para precios inválidos.**

---

## Reto Final

**Reto:**

1. Agregar en **VideojuegoService** el método **buscarPorGenero(String genero)** que retorne todos los videojuegos de un género. Exponerlo como **GET /api/videojuegos/genero?nombre=RPG**.
2. Agregar en **VideojuegoRepository** el método **findByPrecioMenorA(double max)** y exponerlo en el controlador como **GET /api/videojuegos/ofertas?precioMax=150**.
3. Modificar la validación del Servicio para que también rechace títulos vacíos o nulos, lanzando **IllegalArgumentException** con un mensaje descriptivo.
4. Agregar el campo **disponible** (boolean) a la entidad **Videojuego** y crear un endpoint **PATCH /api/videojuegos/{id}/disponibilidad** que solo cambie ese campo sin modificar el resto.

---

## Conclusión

Construimos una API REST aplicando correctamente la arquitectura en capas.

Separamos responsabilidades: entidad, repositorio, servicio y controlador.

Aplicamos inyección de dependencias por constructor y uso de Optional.

Controlamos los códigos de estado HTTP con ResponseEntity y @ExceptionHandler.

**Siguientes pasos: Reemplazar la lista en memoria con Spring Data JPA y una base de datos PostgreSQL.**